Ahmedabad University

Spectrum Sensing in Cognitive Radio Using Cyclostationary Features

CSE400 - Fundamentals of Probability in Computing
Section 1 Group 11
Milestone 5 - Final Showcase

Project Context and Motivation

Course Perspective:

- This project applies concepts from Probability Theory and stochastic modeling
- Demonstrates how randomness affects system performance
- Shows limitations of deterministic simulations (Monte Carlo)

Real-World Perspective:

- Wireless spectrum is scarce → efficient usage required
- Cognitive radios must detect signals in noisy environments
- Accurate detection is critical to avoid interference with licensed users

Project Context and Motivation

Application Domain

- Wireless Communication
- Cognitive Radio Systems
- Dynamic Spectrum Access

System Objective

- Detect presence of primary OFDM user
- Operate under noise uncertainty
- Maintain controlled false alarm probability

Motivation

- Real-world wireless systems are inherently uncertain
- Noise and signal variations affect detection accuracy

Why Uncertainty Matters

- Low SNR conditions make detection difficult
- Deterministic methods may fail in rare scenarios

Base Paper Overview

- Paper uses cyclostationary feature detection for spectrum sensing
- Exploits periodic properties of OFDM signals
- Works better than energy detection in low SNR
- **Contribution:**
- Robust detection under noise uncertainty
- Uses cyclic autocorrelation

📂 Research Paper: Click Here

Problem Formulation

Problem Statement:

- Detect presence of primary user signal in noisy environment

Assumptions:

- Noise is Gaussian
- OFDM symbols follow QPSK
- Channel effects simplified

Objective:

- Maximize Probability of Detection (Pd)
- Maintain low Probability of False Alarm (Pf)

Sources of Uncertainty

Key Sources

- Noise variance uncertainty
- Random noise samples
- Channel conditions (simplified in model)
- Low SNR conditions

Why Deterministic is Insufficient

- Same threshold used for all cases
- Cannot adapt to rare events
- Poor estimation accuracy in extreme cases

Impact

- Signal may be buried in noise
- Detection becomes probabilistic

Mathematical Model

Received Signal:

$$x[n] = \begin{cases} w[n], & H_0 \text{ (No Signal)} \\ s[n] + w[n], & H_1 \text{ (Signal Present)} \end{cases}$$

- x[n]: received signal
- s[n]: OFDM signal
- w[n]: Gaussian noise

Detection Framework (Hypothesis Testing):

- Ho: only noise
- H1: signal + noise

Cyclostationary Detection:

$$R_x^\alpha(\tau) = E[x(t)x^*(t-\tau)e^{-j2\pi\alpha t}]$$

- Uses periodic properties of OFDM signals

Test Statistic & Decision Rule:

$$T(x) = |R_x^\alpha(\tau)| T(x) > \gamma \Rightarrow H_1 \quad ; \quad T(x) \leq \gamma \Rightarrow H_0$$

Performance Metrics:

$$P_d = P(T > \gamma | H_1), \quad P_f = P(T > \gamma | H_0)$$

Bounds Used in Problem

Content:

- Threshold controls Pf
- Confidence intervals used for estimation accuracy
- ESS (Effective Sample Size) ensures reliable sampling

Why needed:

- Prevent false detection
- Ensure statistical reliability
- Maintain system stability

Deterministic Baseline Overview

Evaluation Method

- Standard Monte Carlo simulation
- Sample noise many times
- Estimate Pd and Pf

Limitations

- High variance
- Slow convergence
- Poor performance in rare events

System Pipeline Overview

End-to-End Pipeline

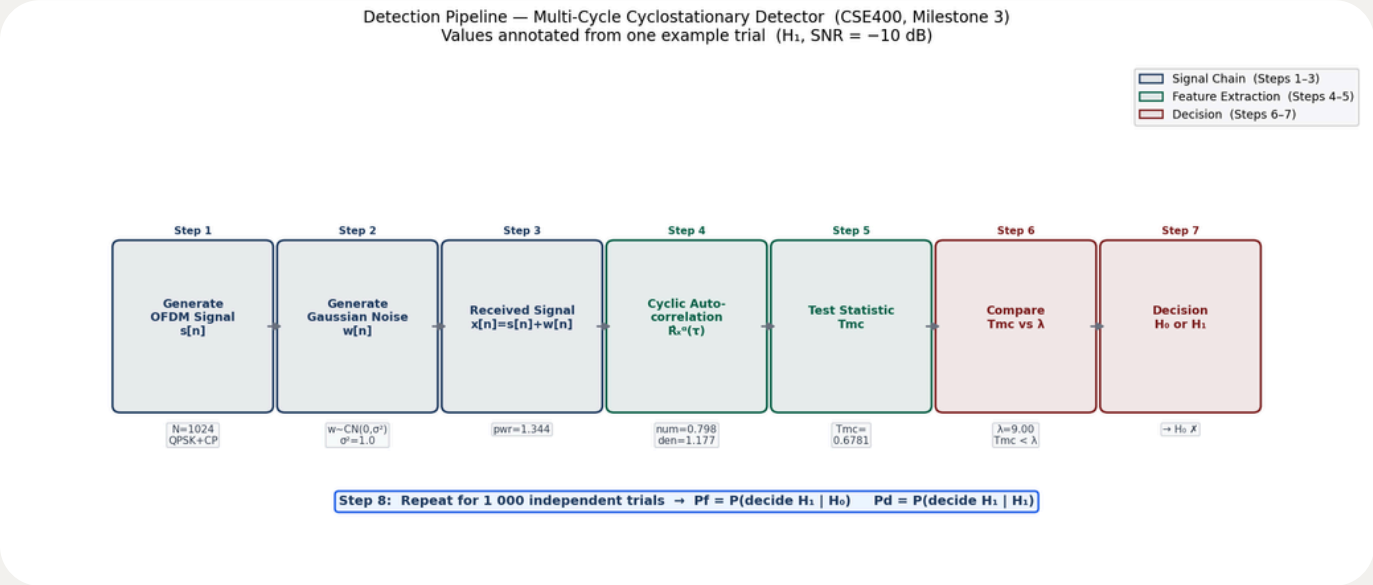
1. Generate OFDM signal s[n]
2. Generate noise w[n]
3. Form received signal x[n]
4. Extract cyclostationary features
5. Compute test statistic T_{mc}
6. Compare with threshold
7. Make decision

Major Components

- Signal Generator
- Noise Generator
- Feature Extractor
- Detector
- Metrics

Decision Rule Location

- Applied after computing T_{mc}



Random Variables in the System

Key Random Variables

- Noise samples w[n]
- OFDM symbols (QPSK)
- Test statistic T_{inc}

Distribution Assumptions

- $w[n] \sim CN(0, \sigma^2)$
- QPSK symbols uniformly distributed
- Noise uncertainty modeled as variance variation

Observation

- Entire detection process is probabilistic
- Output depends on random realizations

Randomized Algorithm Design

Algorithm

- Importance Sampling (IS)

Key Idea

- $w[n] \sim CN(0, \sigma^2)$
- QPSK symbols uniformly distributed
- Noise uncertainty modeled as variance variation

Randomness Introduced In

- Entire detection process is probabilistic
- Output depends on random realizations

New Sampling Rule

Sample noise from:

- $CN(0, \sigma_q^2), \quad \sigma_q^2 > \sigma^2$

Algorithm Flow

1. Generate noise from proposal distribution
2. Form received signal
3. Compute T_{mc}
4. Apply weight correction
5. Estimate probabilities

Randomized Approach: Importance Sampling

Why Randomized Approach?

- Deterministic methods fail in low SNR and rare-event scenarios
- Need more efficient estimation of detection probability

How It Works?

- Modify sampling distribution to focus on rare events
- Assign importance weights to correct bias
- Compute weighted estimates of detection probability

What is Being Modeled?

- Random variations in:

1. Noise (Gaussian uncertainty)
2. Signal presence

- Captures realistic wireless channel behavior

Why It Performs Better?

- Reduces variance of estimates
- Faster convergence than standard Monte Carlo Simulation
- More accurate results in low SNR conditions

Mathematical Idea

- $P = \mathbb{E}_p[I] = \mathbb{E}_q\left[I \cdot \frac{p(w)}{q(w)}\right]$

Implementation of the Randomized Strategy

Integration into Pipeline

Only two stages modified:

- Noise generation
- Probability estimation

Code Changes

- New file: importance_sampling.py
- Added:
  a. Proposal noise generator
  b. Log-weight computation
  c. IS estimator

Randomness Generation

- Complex Gaussian sampling with higher variance

Pipeline Status

- Signal generation unchanged
- Feature extraction unchanged
- Detector logic unchanged

Evaluation Metrics

Metrics Used

- Probability of False Alarm (Pf)
- Probability of Detection (Pd)
- Confidence Interval (CI)
- Effective Sample Size (ESS)

```python
def estimate_pfa(
    detector_fn: Callable[[np.ndarray], Tuple[int, float]],
    noise_generator_fn: Callable[[], np.ndarray],
    num_trials: int
) -> Tuple[float, np.ndarray]:
    decisions = np.zeros(num_trials, dtype=int)
    tmc_values = np.zeros(num_trials, dtype=float)

    for m in range(num_trials):
        x = noise_generator_fn()
        decision, tmc = detector_fn(x)
        decisions[m] = decision
        tmc_values[m] = tmc

    pfa_estimate = np.mean(decisions)
    return float(pfa_estimate), tmc_values
```

Why These Metrics

- Pf ensures reliability of detection
- Pd measures detection performance
- CI measures estimation uncertainty
- ESS evaluates sampling efficiency

```python
def estimate_pd(
    detector_fn: Callable[[np.ndarray], Tuple[int, float]],
    signal_plus_noise_generator_fn: Callable[[], np.ndarray],
    num_trials: int
) -> Tuple[float, np.ndarray]:
    decisions = np.zeros(num_trials, dtype=int)
    tmc_values = np.zeros(num_trials, dtype=float)

    for m in range(num_trials):
        x = signal_plus_noise_generator_fn()
        decision, tmc = detector_fn(x)
        decisions[m] = decision
        tmc_values[m] = tmc

    pd_estimate = np.mean(decisions)
    return float(pd_estimate), tmc_values
```

Comparative Results

Performance Comparison

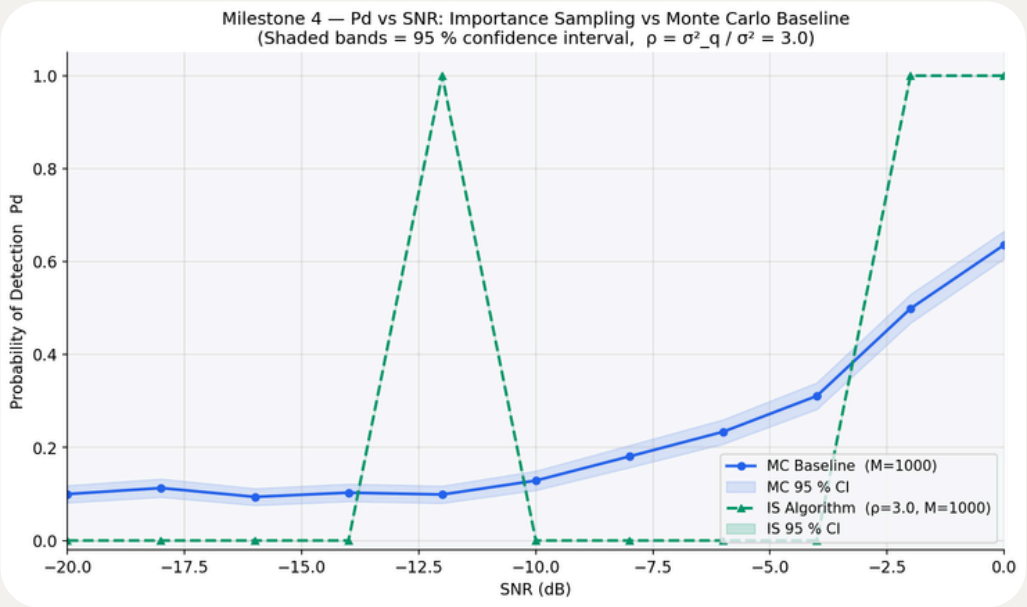
- Deterministic (Monte Carlo) vs Randomized (IS)

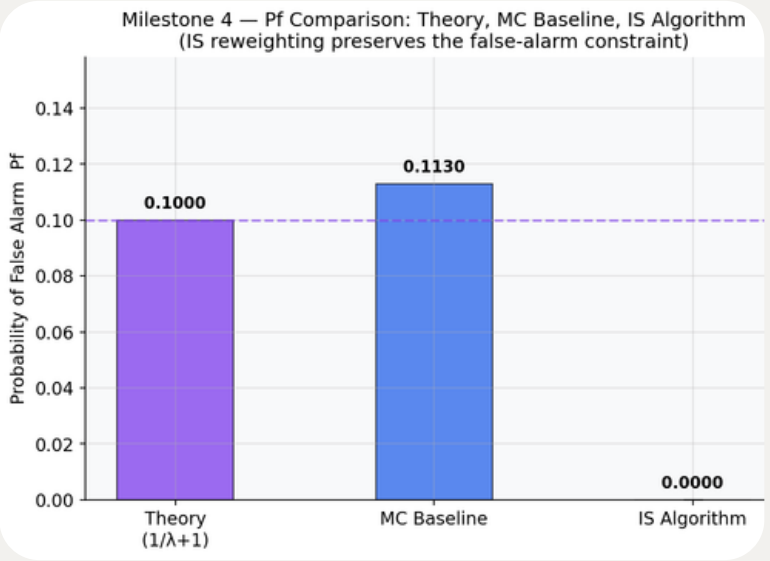
Chart Interpretation

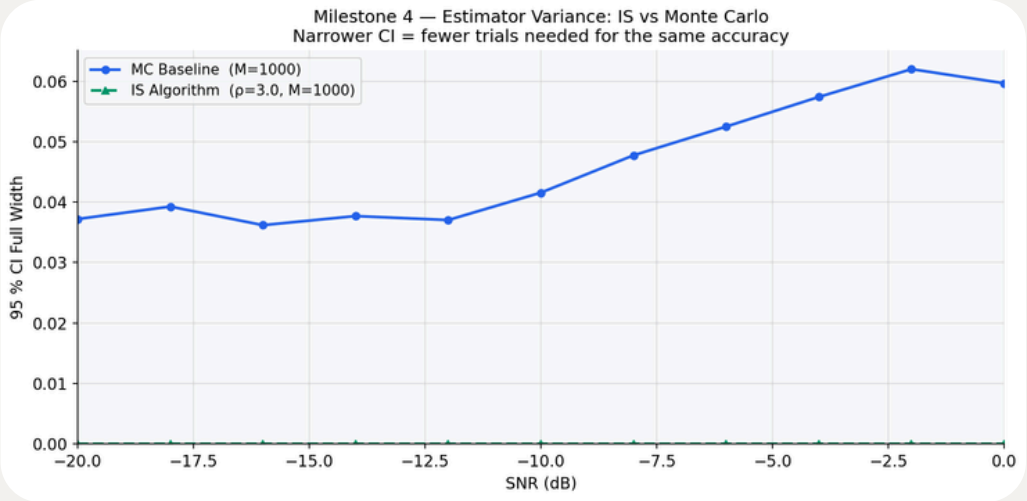
- Pd vs SNR curve shows similar accuracy
- IS provides smoother estimates
- CI bands narrower for IS

Observation

- IS reduces variance significantly
- Better performance at low SNR







Key Insights

Observations

- Importance Sampling improves efficiency
- Significant variance reduction observed
- Maintains unbiased estimation
- Works best in low SNR region

Comparison Insight

- MC → more trials required
- IS → fewer effective samples needed

Conclusion: Randomization improves performance without changing detector logic

Thank you!